Culprit: Product Requirements Document

Oct 19, 2026 · @Ankesh

1. Overview

Culprit answers one question in under ten seconds: **which commit most likely broke production?** You give it an error log and the project's git history. It returns a ranked list of suspect commits, the evidence behind each ranking, and an interactive timeline that shows the failure and the changes that landed just before it.

Working name: Culprit. **Buildathon track:** Systems, Infrastructure & Cybersecurity. **Format:** a web app with a small CLI helper, built solo in about 37 hours.

What the top of the screen looks like after analysis:

> **Most likely culprit:** `a3f9c21` **"Refactor payment retry backoff"** by Priya, 14 minutes before the first error. It changed `payments/retry.js` lines 41 to 58, which is the top stack frame in 312 of 340 errors. The error signature "TypeError: cannot read 'attempts'" never appears before this commit.

Culprit is a heuristic, not a verdict. It shows a suspicion score and always lists the evidence, so a person can check the reasoning in seconds.

2. Problem and users

When something breaks after a deploy, the first question is always "what changed?" Answering it today means scrolling a log, finding the first error, noting its time, then walking through `git log` and guessing which commit matters. It is slow, and it is hardest at 2am under pressure.

Culprit automates that first pass. It does not fix anything and does not replace judgment. It narrows dozens of commits to a short, explained shortlist.

User	Situation	What they get
Student or hackathon team	App broke after a late-night merge	The commit to look at first, with evidence

User	Situation	What they get
Indie developer	Small service, no monitoring stack	A postmortem-ready summary without setting up tooling
On-call engineer at a small team	Alert fired, many recent deploys	A ranked shortlist to hand to the right person

Primary user for the demo: the developer who has a log file and a repo and no time.

3. Goals, non-goals and success criteria

The goal is a small tool that works, rebuilds cleanly and is obviously useful. The buildathon page says "small and polished beats big and broken", so every scope decision below favours finishing.

Goals

- Name the most likely culprit commit from a log plus git history, with visible evidence.
- Make the verdict understandable at a glance, then explorable (timeline, diff, error groups).
- Work offline, deterministically, so tests pass in Docker with no network.
- Ship a polished frontend and a 90-second demo that needs no explanation.

Non-goals (cut without guilt)

- Live log streaming, accounts, databases, cloud storage.
- Machine learning or any runtime AI call. The ranking is plain, explainable code.
- Support for more than two log formats and two stack-trace styles.
- Auto-fixing, rollbacks, or integrations with GitHub, Slack or PagerDuty.

Success criteria, mapped to the four judging criteria

Judging criterion	Culprit's answer	How to prove it
It works	Bundled sample incidents give a correct verdict in one click	README steps plus a demo video
It's worth using	Solves a real, familiar problem; answer in under 10 seconds	The verdict card and a copyable postmortem

Judging criterion	Culprit's answer	How to prove it
It rebuilds	One Dockerfile, lockfile, pinned Node, offline `npm test`	Run `docker build` then the test with `--network none` yourself
OmniRush built it	All code written in listed sessions	Focused sessions, sub-agents, tests written alongside

4. How it works

Culprit turns a log and a commit list into a ranked suspect list in five steps, all deterministic.

1. **Parse the log.** Detect the format (JSON lines, or plain text with an ISO timestamp and level), pull out error events, and attach any multi-line stack trace to its event.
2. **Find the incident window.** Compute the error rate per minute against a baseline from the quiet start of the log. The first sustained rise above baseline is the *onset*. Everything after it is the incident.
3. **Group errors by signature.** Normalise each message by stripping numbers, ids and quoted values, so "user 481 not found" and "user 77 not found" become one signature. Record when each signature first appeared.
4. **Parse the git history.** Read commits with author, time, message, files touched, and changed line ranges (from `git log -p -U0`).
5. **Score every commit before the onset** and rank them.

The score. Each commit before the onset gets four sub-scores between 0 and 1, combined with fixed weights:

```
score = 0.35 * proximity + 0.30 * traceMatch + 0.25 * novelty + 0.10 * churn
```

Signal	What it measures	Rule of thumb
Proximity	How soon before the onset the commit landed	Decays exponentially; a commit 10 minutes before beats one 10 hours before

Signal	What it measures	Rule of thumb
Trace match	Whether the commit touched files and lines in the stack traces	Highest when the changed lines contain the top stack frame of many errors
Novelty	Whether the dominant error signature first appears after the commit and never before	1 if brand new after the commit, 0 if it existed before
Churn	How large and risky the change was	Small bonus for big diffs and for touching many files

The weights are a starting point. Tune them against the bundled sample incidents until the known culprit ranks first, then freeze them so tests are stable.

Honesty rules. The score is a *suspicion score*, never a probability. The UI shows the top three suspects, not one, so a wrong top pick is still visible. If the log has no stack traces, trace match is skipped and the verdict card says confidence is lower and why.

Output. A verdict (top suspect plus evidence), the ranked list, the error-rate series, the commit list for the timeline, the error-signature groups, and a copyable Markdown postmortem.

5. Features

Build in this order. Do not start a "should have" until every "must have" works and is tested.

Priority	Feature	Done when
Must	Load a log file (drag and drop) and detect its format	JSON lines and plain text both parse; bad input shows a clear error
Must	Load git history	Works from a bundled sample and from one documented command's output
Must	Incident onset detection	Finds the same onset every run on the sample logs
Must	Ranked suspects with evidence	Known culprit ranks first on both sample incidents

Priority	Feature	Done when
Must	Verdict card	Shows top suspect, time gap, files and lines, and the matching error count
Must	Timeline	Error-rate chart with commits underneath; suspect highlighted
Should	Suspect detail drawer	Diff with the lines that match stack frames highlighted
Should	Error signature table	Groups, counts, first-seen time, click to filter the timeline
Should	"Try a sample" button	One click loads a bundled incident, so judges see it work instantly
Should	Copy as Markdown	A short postmortem draft with the verdict and evidence
Nice	Dark and light theme	Follows system setting
Nice	Keyboard shortcuts	Arrow keys step through suspects

Cut list: live tailing, uploading whole repositories in the browser, more log formats, user settings for the weights, sharing links, and anything that needs the network.

6. Screens and UX

The verdict is the hero. Everything else supports it. A visitor should understand the product within five seconds of the results screen appearing.

Screen 1: Start. A single drop zone for the log, a secondary slot for git history, and three large "Try a sample incident" buttons. Nothing else on the page.

Screen 2: Verdict. A large card with the top suspect's short hash, message, author and time gap, then three evidence lines with numbers. Below it, two smaller cards for suspects two and three. A clear "copy postmortem" button.

Screen 3: Timeline. An error-rate area chart across the top, with the onset marked. Under it, a lane of commit markers aligned to the same time axis, the top suspect glowing. Drag to zoom a time window. Click a spike to see the log lines behind it in a side panel.

Screen 4: Suspect drawer. Slides over the right side. Shows the commit's diff, with lines that overlap stack frames highlighted, and a plain list of why it ranked where it did (one line per signal, with its score).

Screen 5: Error groups. A table of signatures with count, first seen and a small sparkline. Clicking a row filters the timeline.

Design direction

- **Mood:** a calm "incident console". Near-black background, one hot accent colour reserved only for the culprit and the onset, so the eye always lands there.
- **Type:** one clean sans for the interface and one monospace for hashes, paths and log lines. Bundle the fonts locally so nothing depends on the network.
- **Motion:** purposeful only. The verdict card animates in once, the timeline highlights the suspect, drawer slides. No decorative animation.
- **States:** design the empty, loading, error and "low confidence" states on purpose. Judges will try a bad file.
- **Responsive:** usable at laptop width first; readable on a phone for the live link.
- **Accessibility:** colour is never the only signal; keyboard focus is visible; contrast passes.

Workflow tip for the frontend: ask the agent to define a small design system first (colours, type scale, spacing, components), build each screen against it, then refine from screenshots rather than describing problems in words.

7. Technical design

The design splits a pure, testable core from a thin user interface. All analysis runs in the browser, so the hosted demo needs no server and the tests need no network.

Part	Responsibility	Notes
Core library	Log parsing, onset detection, signatures, git parsing, scoring	Pure TypeScript functions, no I/O, fully unit-tested
Web app	The five screens	Vite plus a UI framework you are comfortable with; custom SVG for the timeline to avoid heavy chart libraries
CLI helper	Runs one git command and writes a JSON export of history	Lets a user analyse a real local repo; optional for the demo

Part	Responsibility	Notes
Fixtures	Sample logs and sample git history as JSON	Committed to the repo; used by tests and the demo buttons

Stack suggestion: TypeScript on Node 22, Vitest for tests, Vite for the web build. Commit the lockfile and an `.nvmrc`. This is a suggestion, not a requirement; let the agent propose alternatives and decide quickly.

Key data shapes

- *Event:* time, level, message, optional stack frames (file, line, function).
- *Commit:* hash, author, time, message, and a list of changed files, each with changed line ranges.
- *Verdict:* onset time, ranked suspects, each with a score, the four sub-scores and evidence lines.

Constraints that shape the design

- Analysis of a 50 MB log must not freeze the page: parse in a web worker and cap the hosted demo at a fixed line count.
- Commit times and log times may use different time zones; normalise everything to UTC and say so in the UI.
- The core must produce identical output for identical input, with no random numbers and no dependence on the current time.
- Never send a user's log or repo anywhere; say "runs entirely in your browser" on the start screen.

8. Testing, Docker and submission

Judges rebuild the project from scratch in Docker and run its tests offline, so this section is as important as any feature. Set it up in the first two hours and keep it green.

Tests that matter most

- Log parsing: each supported format, a malformed file, an empty file, a log with no errors.
- Onset detection: a clean spike, a slow ramp, and no incident at all.
- Signature grouping: different ids and numbers collapse to one signature.

- Scoring: on both sample incidents the known culprit ranks first; a decoy commit that touched the same file long before the incident ranks lower.
- Determinism: the same input twice gives byte-identical output.

Rebuild checklist (from the official buildathon page)

- [ ] Public GitHub repo on the account you joined with, with frequent commits
- [ ] Lockfile committed
- [ ] Node version pinned in `.nvmrc`
- [ ] Dockerfile that installs, builds and runs the app
- [ ] One test command (`npm test`) that works with no internet
- [ ] README: what it is, setup, run, test, screenshot, live link
- [ ] `.env.example` present, with no real secrets anywhere
- [ ] No paths from your own computer in the code

Self-check like a judge

```
docker build -t culprit . && docker run --rm --network none culprit npm test
```

Rules to remember

- OmniRush must be your only generative AI tool for the project, for code, text and images. Turn off AI autocomplete in your editor.
- Start fresh during the event; code not written in your listed sessions can be flagged.
- This PRD is for your own understanding. Do not paste it into the repo as your own documentation. Have OmniRush write the repo's README and notes inside your sessions.
- Submit by 20 Oct, 11:59 PM IST. Submit an early version first; you can resubmit with the same ID.

9. Demo data and demo script

The demo is only as good as the sample incidents, so invent them carefully. They also serve as your test fixtures.

Sample incidents (all fictional, bundled in the repo)

Incident	Story	What it shows
Checkout outage	A retry-logic refactor breaks payments 14 minutes before errors spike	The clear case: stack frames point straight at the culprit
Slow-burn failure	A config change causes a gradual rise in timeouts	The onset is a ramp, not a spike
Ambiguous case	Two plausible commits; the real one is not the closest in time	Shows honesty: top three suspects and lower confidence

Each sample needs about 30 commits of plausible history, including a few decoy commits that touch the same files earlier, and a log of a few thousand lines with a quiet period before the failure.

90-second demo script

1. **0 to 10 seconds.** The problem in one sentence: "Production broke. Which of these 30 commits did it?"
2. **10 to 25 seconds.** Click a sample incident. The verdict card appears with the culprit and its evidence.
3. **25 to 50 seconds.** Show the timeline: the error spike, the glowing commit just before it, drag to zoom, click the spike to see the real log lines.
4. **50 to 65 seconds.** Open the suspect drawer: highlighted lines that match the stack frame, and the one-line reason for each signal.
5. **65 to 80 seconds.** Load the ambiguous incident to show low confidence and three suspects. This builds trust.
6. **80 to 90 seconds.** Copy the postmortem, show the README and the passing Docker test.

10. Risks and fallbacks

The biggest risk is not technical. It is trying to do too much and failing the rebuild check. Each risk below has a pre-decided fallback so you do not lose time deciding under pressure.

Risk	Likelihood	Fallback
Scope creep (more formats, more screens)	High	Re-read the cut list; ship the must-haves polished before anything else
Ranking picks the wrong commit on a sample	Medium	Tune weights against samples, then freeze; show top three so a miss is still visible
Real logs have no stack traces	Medium	Skip trace match, lower the stated confidence, explain why on screen
Time zone mismatches between log and git	Medium	Normalise to UTC; show the assumption on screen
Large files slow the page	Medium	Web worker parsing and a documented size cap for the hosted demo
Docker or offline tests fail late	Medium	Set up and run them in the first two hours, and after every major change
Token budget runs low (5M tokens)	Low to medium	Check usage after the first hours; use short, specific prompts; avoid re-pasting large files; use sub-agents only for truly parallel work
Sessions do not match the code	Low	Build everything inside OmniRush sessions; never paste code from elsewhere

Decision rule for the last 6 hours: if a feature is not finished and tested, cut it. A polished smaller project beats an ambitious broken one.

11. Plan for the time left

About 37 hours remain before the deadline (20 Oct, 11:59 PM IST). The plan uses roughly 30 hours of work and keeps the rest as buffer and sleep. Hours are counted from when you start.

1. **Hours 0 to 2: foundation.** Create the repo, lockfile, pinned Node version, Dockerfile, an empty passing `npm test`, and a README skeleton. Run the Docker offline check once. Make a first submission with just this, so a valid entry exists.

2. **Hours 2 to 5: fixtures.** Create the three sample incidents (logs and git history) and decide the expected culprit for each. These drive everything after.
3. **Hours 5 to 14: core engine.** Log parsers, onset detection, signatures, git parsing, scoring, with tests written alongside. Use sub-agents for parallel pieces (parsers, tests, review). Stop when the known culprit ranks first on all three samples.
4. **Hours 14 to 16: checkpoint.** Run the full Docker offline check. Commit. Resubmit. Take a break or sleep here if you can.
5. **Hours 16 to 20: design system and verdict screen.** Colours, type, spacing, components first; then the start screen and the verdict card.
6. **Hours 20 to 26: timeline and drawer.** Error-rate chart, commit lane, zoom, click-through to log lines, the suspect drawer with highlighted diff lines.
7. **Hours 26 to 29: polish.** Empty, error and low-confidence states, error groups table, copy-as-Markdown, responsive checks, accessibility pass.
8. **Hours 29 to 32: hardening.** Bad files, huge files, time-zone cases. Run the Docker offline check again. Fix anything it finds.
9. **Hours 32 to 35: ship.** Deploy the live link, record the demo video, finish the README with a screenshot, confirm the session list matches your code.
10. **Hours 35 to 37: buffer.** Final resubmit, copy the Submission ID into Unstop. Do not start new features.

Sleep matters. A rested hour at the end is worth more than a tired hour in the middle. Plan at least one real sleep block between hours 14 and 20.